Assignment 3: TCP Congestion Control Implementation

CS 422: Computer Networks, Fall 2026

Group Members: Ayusha Patra, Kabileswaran Krishna Kumar, Vihaan Kodeboyina, Millan Kumar

GitHub Repo link:
https://github.com/LelsersLasers/CS422-Fall2026/tree/main/Assignment3

Selected Option: Option 1: Kernel Module Implementation

Reproducibility / Kernel Setup / Building

- Note: build environment used in development: x86-64 Linux 6.18.50 (NixOS), GCC 15.2.0, and GNU binutils 2.46
- Required kernel features are IPv4/TCP (CONFIG_INET), loadable modules (CONFIG_MODULES), advanced TCP congestion control (CONFIG_TCP_CONG_ADVANCED), CUBIC (CONFIG_TCP_CONG_CUBIC), and BBR (CONFIG_TCP_CONG_BBR). Root privileges are needed to load/unload modules

TO BUILD

Run from the repository root:

- sudo apt-get update
- sudo apt-get install build-essential kmod python3-venv "linux-headers-$(uname -r)"
- python3 -m venv .venv
- source .venv/bin/activate
- python3 -m pip install "matplotlib==3.10.5" "numpy==2.3.4"

CHECK VERSIONS

- uname -r
- gcc --version
- ld --version
- python3 --version
- python3 -m pip freeze

Compile and load RLA (RLA = RTT–Loss AIMD)

Run from repo root:

- make -C Assignment3/kernel clean
- make -C Assignment3/kernel
- modinfo -F vermagic Assignment3/kernel/tcp_rla.ko
- sudo insmod Assignment3/kernel/tcp_rla.ko
- sudo modprobe tcp_bbr
- cat /proc/sys/net/ipv4/tcp_available_congestion_control

The vermagic kernel release must match uname -r, and the available-algorithms list must include rla, cubic, reno, and bbr. The Makefile uses /lib/modules/$(uname -r)/build. If the matching kernel development tree is elsewhere, pass KDIR=/path/to/kernel/build. Make sure RLA is loaded throughout the experiments.

Run all experiments / generate the plots

- MPLBACKEND=Agg python3 Assignment3/main.py --destinations 3 --repetitions 3 --duration 10 --interval 0.2 --seed 42 --output Assignment3/output/evaluation

This selects three reachable destinations using preliminary CUBIC tests, then attempts 36 comparison trials: four algorithms, three destinations, and three repetitions. All comparison trials use the same destinations, 10-second duration, and 0.2-second sampling interval. The order is RLA/CUBIC/Reno/BBR, CUBIC/Reno/BBR/RLA, then Reno/BBR/RLA/CUBIC. The client selects congestion control on the data socket before connecting and verifies the algorithm actually used. You should likely choose a new output directory for each rerun.

The output directory contains trials.csv, manifest.json, aggregate.csv, and comparison.pdf. Raw per-interval measurements (*_samples.csv) and server responses (*_server_results.json) are stored under run_01/, run_02/, and run_03/, grouped by algorithm. Preliminary selection measurements are under selection/ and are excluded from comparison statistics. The manifest records the kernel, selected destinations, parameters, trial order, and failures. Only destinations with successful trials for every algorithm and repetition contribute to aggregate comparisons.

Clean up

- sudo rmmod tcp_rla

Relevant code: Assignment3/kernel/Makefile, lines 1–12; Assignment3/README.md; Assignment3/main.py, lines 39–96 and 115–251; Assignment3/plot_evaluation.py, lines 22–123; Assignment2/part1/run_test.py, lines 151–171.

- https://github.com/LelsersLasers/CS422-Fall2026/blob/main/Assignment3/kernel/Makefile#L1-L12
- https://github.com/LelsersLasers/CS422-Fall2026/blob/main/Assignment3/README.md
- https://github.com/LelsersLasers/CS422-Fall2026/blob/main/Assignment3/main.py#L115-L251
- https://github.com/LelsersLasers/CS422-Fall2026/blob/main/Assignment3/plot_evaluation.py#L22-L123
- https://github.com/LelsersLasers/CS422-Fall2026/blob/main/Assignment2/part1/run_test.py#L151-L171

1a) Congestion Control Implementation

Our RTT–Loss AIMD (RLA) controller is implemented in Assignment3/kernel/tcp_rla.c. The window-update rules are in rla_on_ack (lines 202–299), throughput utilization is checked in rla_busy (lines 146–166), and Linux callbacks and module registration are defined in lines 306–452.

Initial state and parameters: rla_init clears the per-connection history while preserving Linux's initial congestion window and slow-start threshold (lines 306–312). The baseline RTT starts unknown and becomes the lowest positive smoothed RTT observed. Each control interval lasts max(100 ms, RTT at interval start), with the starting time, acknowledged-byte count, cwnd, MSS, and RTT saved together (lines 174–190). cwnd is measured in segments. RLA respects Linux's window clamp; its normal backoff floor is two segments, although Linux can temporarily use one after a timeout.

RTT and window growth: While cwnd is below the slow-start threshold and the sender is cwnd-limited, RLA increases cwnd by newly acknowledged segments, capped by the threshold and window clamp. At a completed interval, RTT above 1.25 times the baseline takes priority: RLA sets cwnd to max(2, floor(0.90 × cwnd)), subject to the clamp, and sets the slow-start threshold to that window. After slow start, RLA adds one segment per completed interval only when RTT is below 1.10 times the baseline, the sender is cwnd-limited, and the throughput check passes. Otherwise it holds cwnd (lines 238–299).

Throughput: RLA measures acknowledged goodput as 8 × newly acknowledged bytes / actual elapsed seconds. Growth requires this to be at least 80% of the interval-start window rate, 8 × starting cwnd × starting MSS / starting RTT in seconds. rla_busy evaluates the equivalent integer inequality with overflow checks (lines 146–166). Throughput gates growth; it is not treated as a direct estimate of link capacity.

Loss feedback: RLA uses Linux's native congestion notifications as its loss/congestion signal. rla_ssthresh requests max(2, floor(cwnd / 2)) (lines 97–100 and 323–327). Linux

applies recovery, retransmission, and timeout behavior; RLA does not separately halve cwnd for every retransmitted packet. Recovery and idle transitions invalidate the current interval while retaining the RTT baseline (lines 334–365). The ACK callback reads Linux's RTT and acknowledged-byte counters and applies RLA's decisions outside recovery (lines 377–420).

There are no changes from Assignment 2, only implementation additions. This is due to updating the Assignment 2 pseudo code as improvements were made (they are due on the same day). The implementation adds integer arithmetic with overflow checks, counter-validity checks, Linux's spurious-congestion undo callback, and explicit handling of recovery/idle transitions. These details integrate the design with Linux's TCP state machine without introducing another loss backoff.

Socket selection and verification: Assignment2/part1/run_test.py, lines 151–171, creates the data socket and calls setsockopt(IPPROTO_TCP, TCP_CONGESTION, "rla") before connecting. After connecting, get_congestion_algo queries TCP_CONGESTION using getsockopt (Assignment2/part1/tcp_stats.py, lines 144–158). A mismatch raises ProtocolError. The verified algorithm is recorded in the sample CSV's algorithm field and in the trial results. RLA registers under the name "rla" and permits unprivileged per-socket selection (Assignment3/kernel/tcp_rla.c, lines 423–445).

- https://github.com/LelsersLasers/CS422-Fall2026/blob/main/Assignment3/kernel/tcp_rla.c#L97-L299
- https://github.com/LelsersLasers/CS422-Fall2026/blob/main/Assignment3/kernel/tcp_rla.c#L306-L452
- https://github.com/LelsersLasers/CS422-Fall2026/blob/main/Assignment2/part1/run_test.py#L151-L171
- https://github.com/LelsersLasers/CS422-Fall2026/blob/main/Assignment2/part1/tcp_stats.py#L144-L158

1b) Experimental Setup

- We used main.py (lines 115–202) to automatically compare RLA, CUBIC, Reno, and BBR across 3 reachable iPerf3 servers.
- Each algorithm was tested 3 times per destination for 10 seconds, with TCP statistics sampled every 0.2 seconds. The algorithm order was rotated between repetitions to reduce bias (main.py lines 39–67, 140–143).
- We recorded cwnd, RTT, goodput, and retransmissions. Failed trials were tracked, and only complete results were used for comparisons (main.py lines 146–180, plot_evaluation.py lines 37–46).

1c) TCP Statistics Visualizations

- plot_evaluation.py (lines 77–118) generates comparison.pdf containing time-series and scatter plots for all four algorithms using consistent axis scales.

- Our representative destination was speedtest.mia11.us.leaseweb.net:5202, repetition 1.

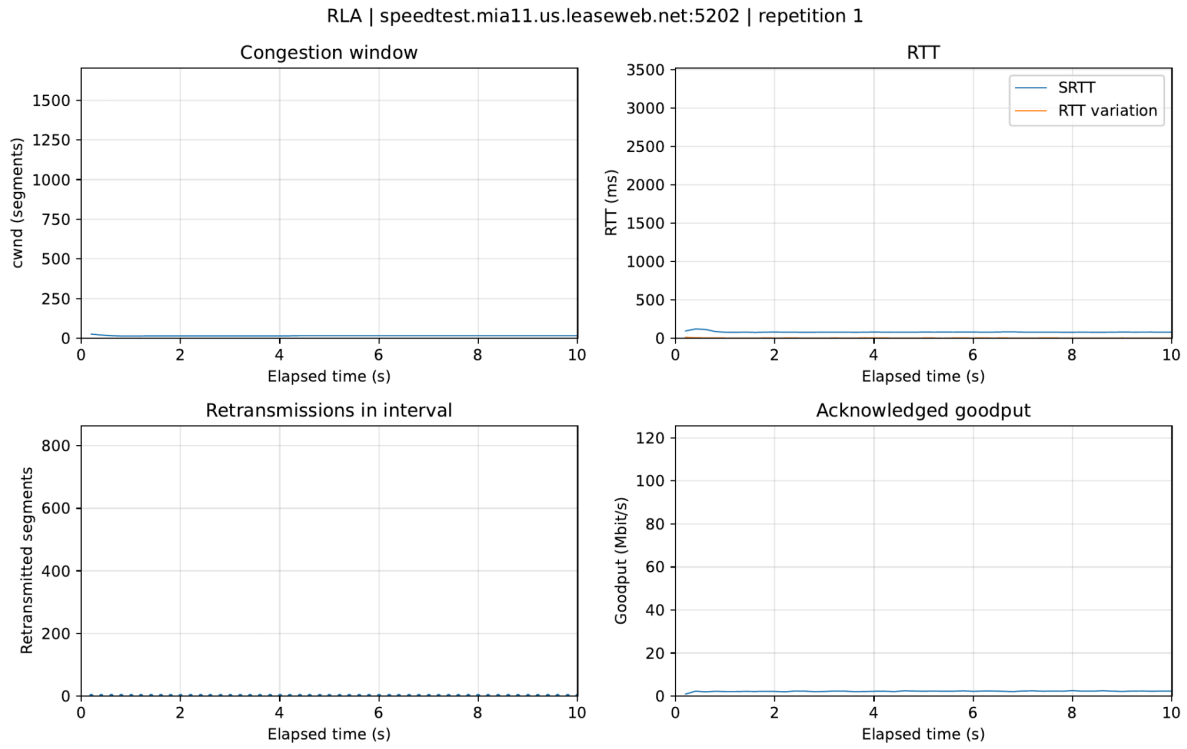


Figure 1: RLA time-series plots



Figure 2: RLA scatter plots

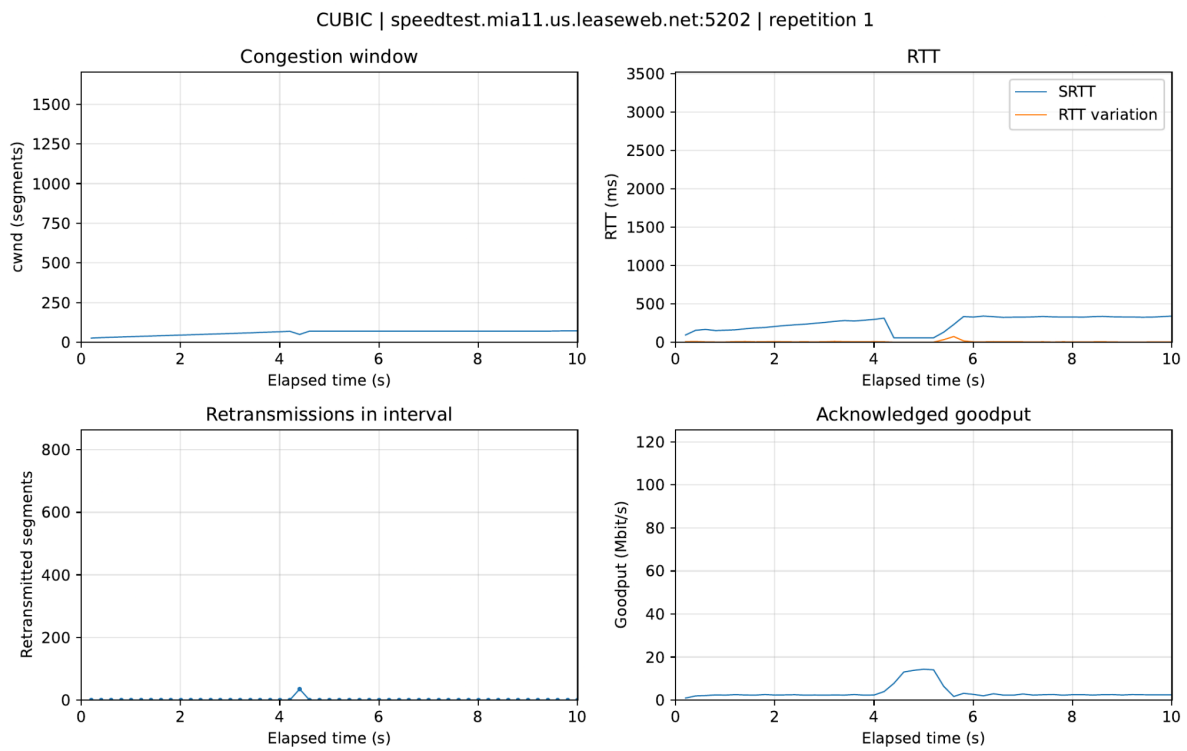


Figure 3: CUBIC time-series plots

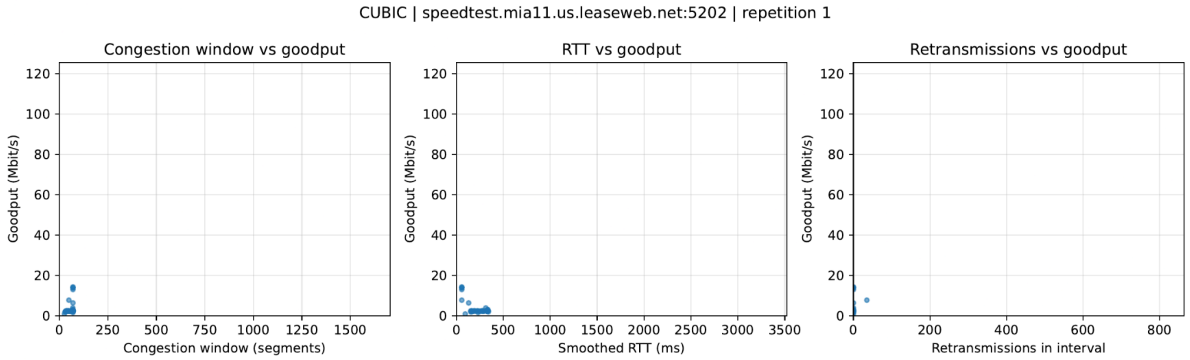


Figure 4: CUBIC scatter plots

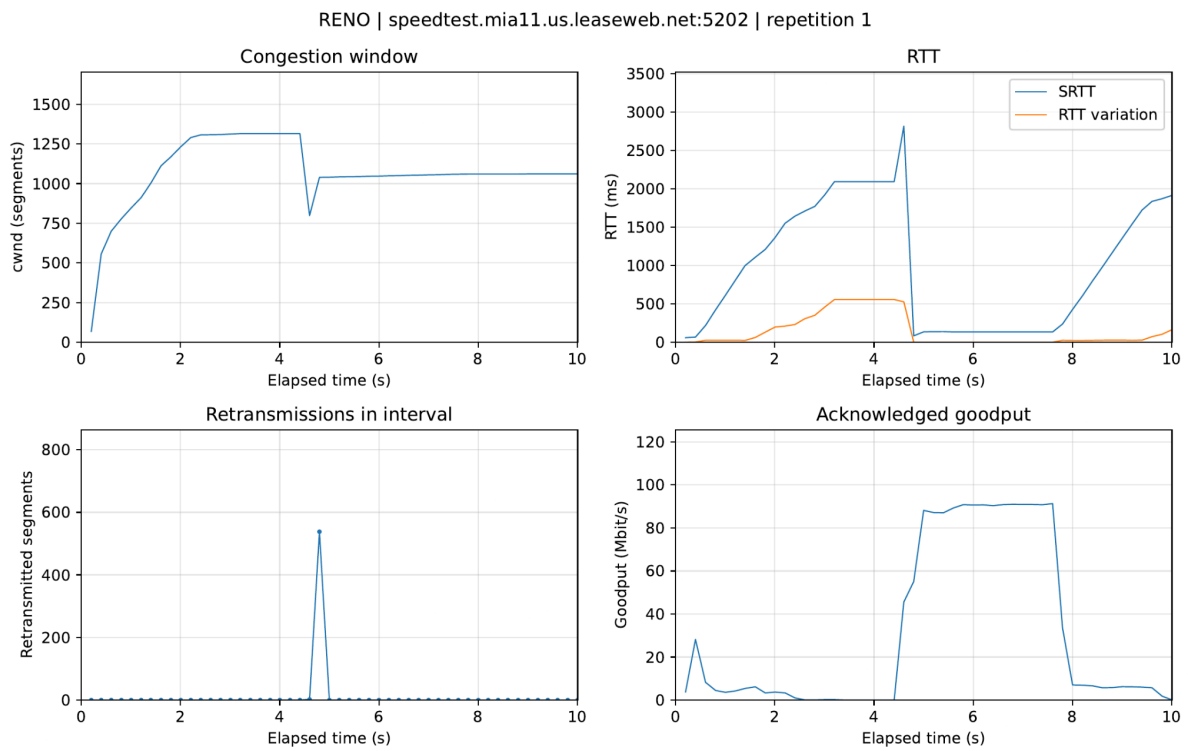


Figure 5: Reno time-series plots



Figure 6: Reno scatter plots

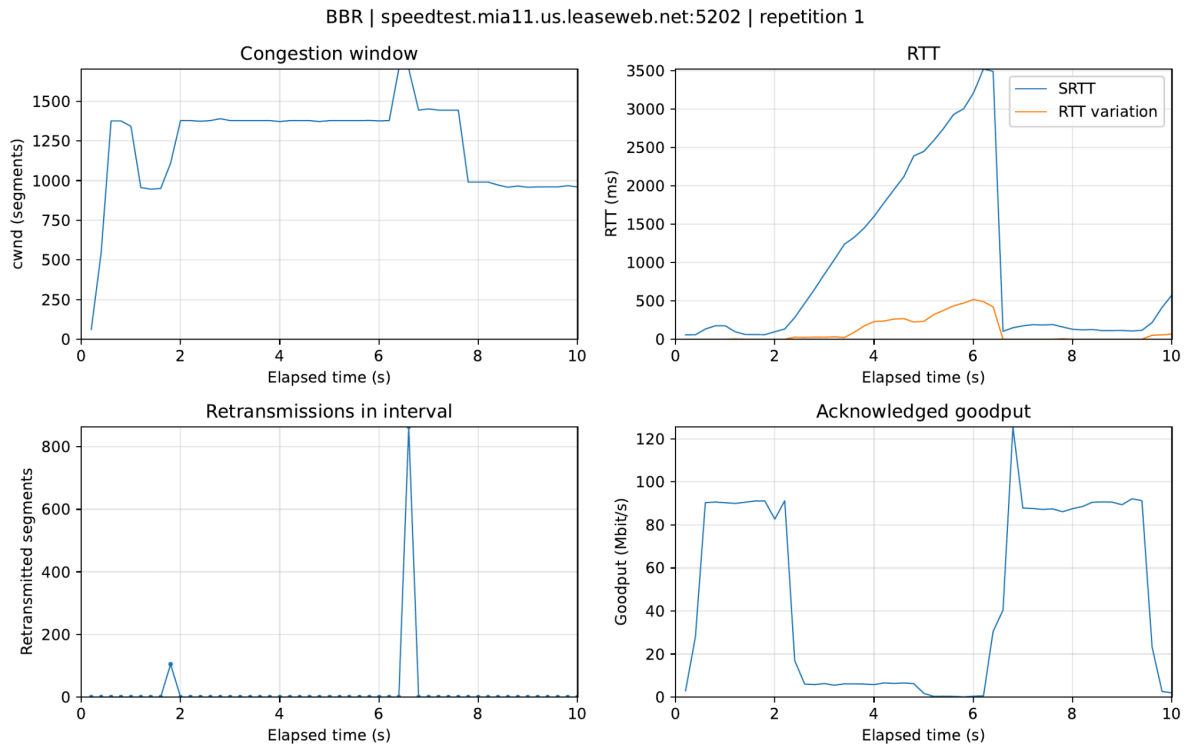


Figure 7: BBR time-series plots



Figure 8: BBR scatter plots

Additional RLA Detail Plots:

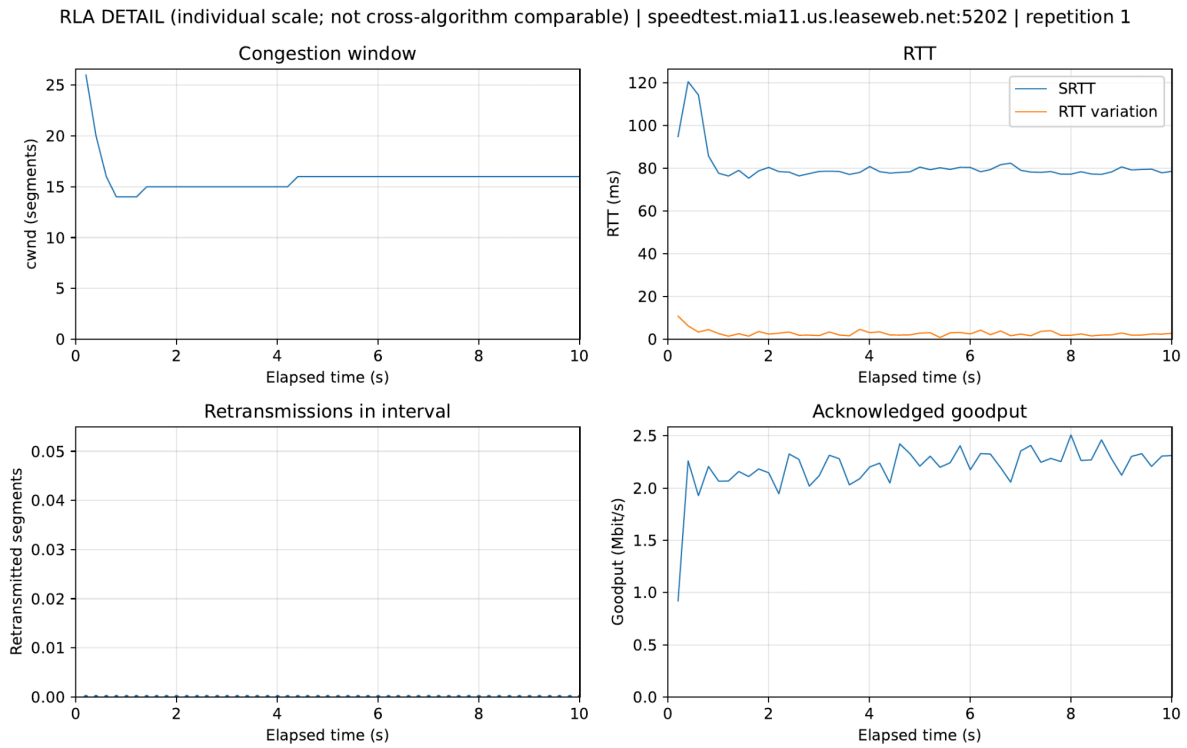


Figure 9: RLA detailed time-series plots (individual scale)

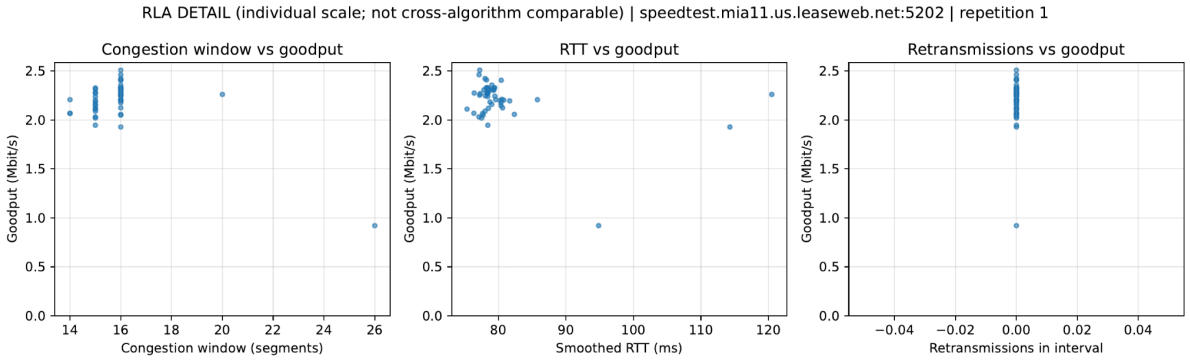


Figure 10: RLA detailed scatter plots (individual scale)

1d) Aggregate Statistics and Observations

- main.py (lines 101–112) calculates RTT and retransmission statistics, while plot_evaluation.py (lines 55–75) generates aggregate.csv containing goodput, RTT, and retransmission statistics across repetitions.

[Insert aggregate statistics table from aggregate.csv.]

Observations:

- **RLA:** Maintained stable goodput around 2–2.5 Mbit/s and RTT near 80 ms, with no retransmissions observed. However, throughput was relatively low.

- **CUBIC:** Gradually increased cwnd, with goodput peaking around 14 Mbit/s and RTT reaching approximately 350 ms.
- **Reno:** Achieved goodput near 90 Mbit/s but experienced large RTT increases (up to approximately 2800 ms) and a retransmission spike.
- **BBR:** Also reached goodput near 90 Mbit/s, but experienced significant RTT fluctuations (up to approximately 3500 ms) and retransmissions.

Overall, RLA showed more stable latency and no observed retransmissions in the representative run, but achieved lower goodput. Reno and BBR achieved higher peak goodput at the cost of greater variability. These observations are based on the representative run; aggregate statistics are needed to assess performance across repetitions.

AI Usage

GPT-6.1 Sol + Codex, qwen3.8:27b, and qwen3:4b, were used to help with this assignment. The LLMs/agents helped critique/validate the algorithm design against collected data, actually connect our algorithm to the Linux Kernel, and help reuse a lot of our Assignment 2 code in our data collection system. They also did a final pass over this report, editing for consistency, grammar, and wording.